MUSTAFACODE

Lecture 13

Pointer to Objects & Date Class (Detailed Notes)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 13: Pointer to Objects & Date Class (Detailed Notes)

📘 Lecture 13 — Pointer to Objects & Date Class (Detailed Notes)

🔹 13.1 Pointer to Objects

📌 Pointer to Object kya hota hai?

Jaise hum normal variables ka pointer banate hain:

```
int x = 10;
int *ptr = &x;
```

waise hi hum objects ka bhi pointer bana sakte hain.

📌 Syntax

```
ClassName *pointerName;
```

Example:

```
Student *ptr;
```

📌 Real Life Example

Socho:

- Ek ghar = Object
- Ghar ka address = Pointer

Pointer directly object ko hold nahi karta.

Wo sirf object ka address store karta hai.

🔹 Simple Example

```cpp
#include<iostream>
using namespace std;

class Student{

public:

    void show(){

        cout<<"Hello Student"<<endl;
    }
};

int main(){

    Student obj;

    Student *ptr;

    ptr = &obj;

    ptr->show();

}
```

📌 Output

```
Hello Student
```

📌 Explanation

Step 1

```cpp
Student obj;
```

Ek object bana.

Step 2

```cpp
Student *ptr;
```

Object pointer banaya.

Step 3

```cpp
ptr = &obj;
```

Pointer me object ka address store kiya.

Step 4

```
ptr->show();
```

Pointer ke through function call kiya.

🔹 Arrow Operator (->)

📌 Kyu use hota hai?

Jab pointer ke through object members access karte hain tab `->` use hota hai.

📌 Example

```
ptr->show();
```

Ye same hai:

```
(*ptr).show();
```

Lekin `->` zyada easy aur readable hota hai.

🔹 Dynamic Allocation using new

📌 new Operator kya karta hai?

`new` runtime par object create karta hai.

📌 new 2 kaam karta hai

1. Memory allocate karta hai

2. Constructor call karta hai

🔹 Example

```cpp
#include<iostream>
using namespace std;

class Student{

public:

    Student(){

        cout<<"Constructor Called"<<endl;
    }

    void show(){

        cout<<"Hello"<<endl;
    }
};

int main(){

    Student *ptr;

    ptr = new Student;

    ptr->show();

}
```

📌 Output

```
Constructor Called
Hello
```

📌 Explanation

Step 1

```cpp
new Student
```

Memory allocate hui.

Step 2

Constructor automatically call hua.

Step 3

Pointer ne object ka address hold kiya.

🔹 Parameterized Constructor Example

```
#include<iostream>
using namespace std;

class Student{

public:

    Student(string name){

        cout<<"Name: "<<name<<endl;
    }
};

int main(){

    Student *ptr;

    ptr = new Student("Ali");

}
```

📌 Output

```
Name: Ali
```

🔹 Array of Objects using new

```
#include<iostream>
using namespace std;

class Student{

public:

    Student(){

        cout<<"Object Created"<<endl;
    }
};

int main(){

    Student *ptr;

    ptr = new Student[3];

}
```

📌 Output

```
Object Created
Object Created
Object Created
```

📌 Explanation

`new Student[3]`

→ 3 objects create karega.

Har object ka constructor alag call hoga.

🔹 delete Operator

📌 Dynamic memory free karne ke liye use hota hai.

📌 Example

```
delete ptr;
```

📌 Array delete

```
delete[] ptr;
```

⚠️ Important

Agar delete use nahi karenge to:

❌ Memory leak ho sakti hai.

==========

🔹 13.2 Breakup of new Operation

📌 Jab hum `new` use karte hain

```
Student *ptr = new Student;
```

to internally 2 steps hote hain.

🔹 Step 1 — Memory Allocation

Computer RAM me object ke liye space reserve karta hai.

🔹 Step 2 — Constructor Call

Object initialize karne ke liye constructor call hota hai.

📌 Real Life Example

Socho hotel room booking:

Step 1

Room reserve hua.

Step 2

Room setup hua.

==========

🔹 13.3 Case Study — Date Class

📌 Problem Statement

Ek `Date` class banani hai jisme user:

- Day set/get kar sake
- Month set/get kar sake
- Year set/get kar sake
- Date increment kar sake
- Default date set kar sake

🔹 Attributes

📌 Data Members

```
int day;
int month;
int year;
```

🔹 Static Member

```
static Date defaultDate;
```

📌 Static kyu use kiya?

Kyuke default date:

✅ Sab objects ke liye same hoti hai.

📌 Real Life Example

Socho school ka:

```
School Name
```

Sab students ke liye same hota hai.

Isliye static.

🔹 Complete Class Structure

```
class Date{

private:

    int day;
    int month;
    int year;

    static Date defaultDate;

public:

    void setDay(int aDay);

    int getDay() const;

    void setMonth(int aMonth);

    int getMonth() const;

    void setYear(int aYear);

    int getYear() const;

    void addYear(int x);

    bool leapYear(int x) const;

    Date(int aDay, int aMonth, int aYear);

    ~Date();
};
```

🔹 Constructor

📌 Constructor kya hota hai?

Special function jo object create hote hi automatically call hota hai.

📌 Example

```
Date(int aDay, int aMonth, int aYear);
```

🔹 Constructor Definition

```
Date::Date(int aDay, int aMonth, int aYear){

    day = aDay;

    month = aMonth;

    year = aYear;
}
```

📌 Real Life Example

Jaise mobile buy karte waqt:

- RAM
- Storage
- Color

initially set karte hain.

🔹 Destructor

📌 Destructor kya hota hai?

Object destroy hone par automatically call hota hai.

📌 Syntax

```
~Date();
```

📌 Example

```
Date::~Date(){

    cout<<"Destructor Called"<<endl;
}
```

📌 Real Life Example

Jaise hotel room leave karte waqt cleaning hoti hai.

🔹 this Pointer

📌 this kya hota hai?

`this` current object ko point karta hai.

📌 Example

```
this->day = day;
```

📌 Meaning

Current object ke `day` member me value store karo.

🔹 Getter Functions

📌 Getter kya karta hai?

Data return karta hai.

📌 Example

```
int getMonth() const{

    return month;
}
```

🔹 Setter Functions

📌 Setter kya karta hai?

Data set karta hai.

📌 Example

```
void setMonth(int a){

    if(a > 0 && a <= 12){

        month = a;
    }
}
```

📌 Validation

Month sirf:

```
1 → 12
```

tak valid hai.

🔹 Leap Year Function

📌 Leap Year Rules

Leap year tab hota hai jab:

- 4 se divide ho
- 100 se na ho

YA

- 400 se divide ho

📌 Example

```
bool Date::leapYear(int x) const{

    if((x%4 == 0 && x%100 != 0)
        || (x%400 == 0)){

        return true;
    }

    return false;
}
```

📌 Example Years

Year	Leap Year
2024	Yes
2025	No
2000	Yes
1900	No

🔹 addYear Function

📌 Purpose

Year increase karta hai.

📌 Example

```
void Date::addYear(int x){

    year += x;
}
```

📌 Example

Agar:

```
2020
```

me:

```
addYear(2);
```

to:

```
2022
```

ho jayega.

🔹 setDate Function

📌 Purpose

Ek hi function me:

- Day
- Month
- Year

sab set karta hai.

📌 Example

```
void Date::setDate(int aDay,
                   int aMonth,
                   int aYear){

    setDay(aDay);

    setMonth(aMonth);

    setYear(aYear);
}
```

🔹 Static Member Initialization

📌 Static members ko class ke bahar initialize karte hain.

📌 Example

```
Date Date::defaultDate(7,3,2005);
```

📌 Meaning

Default date:

```
7 / 3 / 2005
```

🔷 Complete Main Example

```
#include<iostream>
using namespace std;

class Date{

private:

    int day;
    int month;
    int year;

public:

    void setDate(int d, int m, int y){

        day = d;
        month = m;
        year = y;
    }

    void show(){

        cout<<day<<"/"<<month<<"/"<<year;
    }
};

int main(){

    Date obj;

    obj.setDate(20,10,2011);

    obj.show();
}
```

📌 Output

```
20/10/2011
```

🔥 Important Exam Points

📌 Pointer to Object

```
Student *ptr;
```

📌 Access Members

```
ptr->show();
```

📌 Dynamic Object

```
new Student;
```

📌 Delete Object

```
delete ptr;
```

📌 Static Member

Shared by all objects.

📌 Constructor

Automatically call on object creation.

📌 Destructor

Automatically call on object destruction.

📌 this Pointer

Points to current object.

📌 Getter

Returns data.

📌 Setter

Sets data.

📌 Leap Year Logic

```
(x%4 == 0 && x%100 != 0)
|| (x%400 == 0)
```

✅ End of Lecture 13 Notes